



南开大学
Nankai University

面向复杂真实场景的高效多模态视觉感知 从长视频、三维空间到端侧部署

刘云



汇报 提纲

—
CONTENTS

背景与挑战

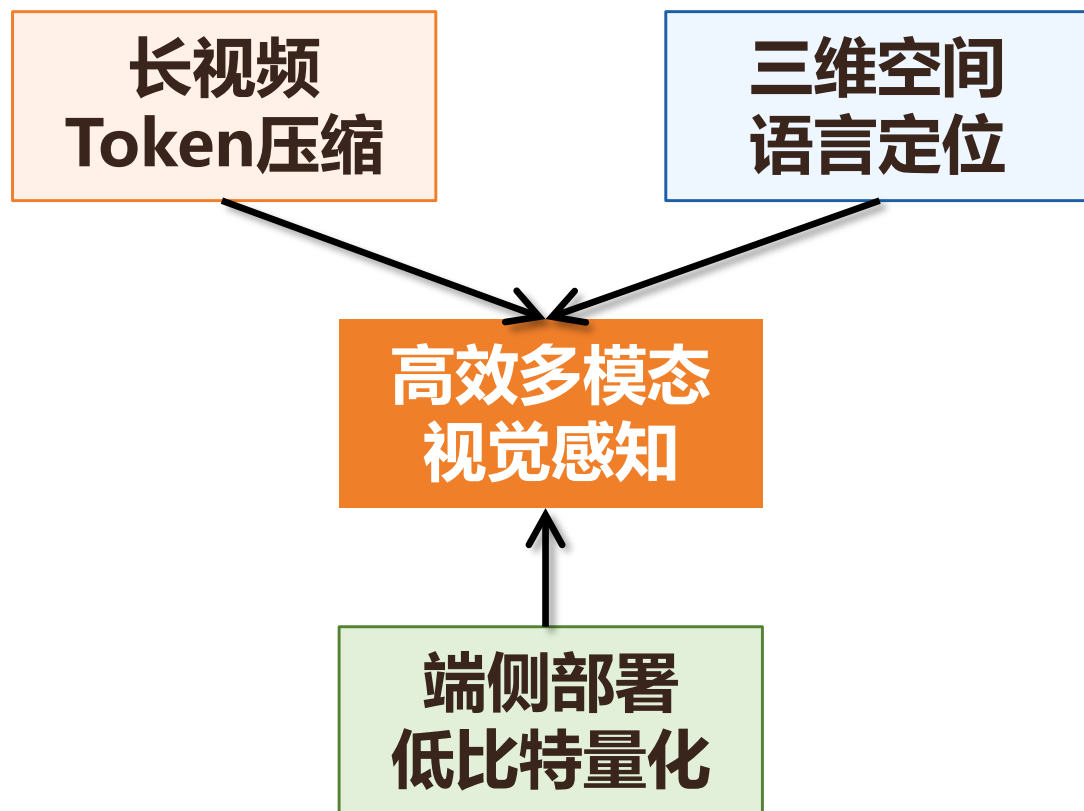
长视频时空压缩_{STAC}

三维空间推理_{VLM-Loc}

端侧低比特部署_{COD-TDQ}

面向复杂真实场景的高效多模态视觉感知

- 从长视频、三维空间到端侧部署



面向复杂真实场景的高效多模态视觉感知

• 复杂真实场景下，多模态视觉感知面临什么问题？



三项工作分别从长时序建模、三维空间理解和端侧部署三个角度展开。

面向复杂真实场景的高效多模态视觉感知

• 从复杂场景挑战到高效多模态感知



以结构感知建模、空间推理与高效部署为主线，推动多模态视觉模型在真实场景中的可靠应用。

汇报 提纲

—
CONTENTS

背景与挑战

长视频时空压缩_{STAC}

三维空间推理_{VLM-Loc}

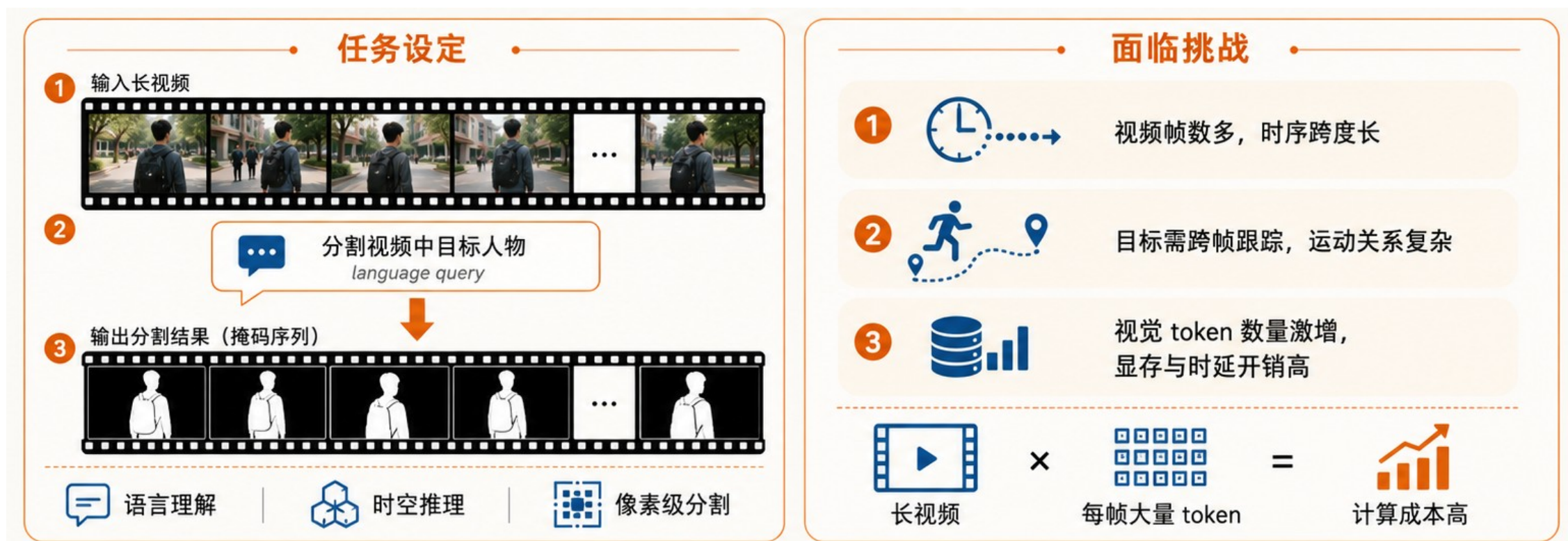
端侧低比特部署_{COD-TDQ}

S. Hesham, Y. Liu, G. Sun, J. Yang, H. Ding, X. Geng, and X. Jiang. STAC: Selective Spatiotemporal Aggregation and Compression for Video Reasoning. ECCV 2026.

STAC: 面向长视频推理的时空 Token 压缩

• 长视频推理分割：理解、跟踪与分割

- 需要同时完成语言理解、时空推理、目标跟踪和像素级分割
- 在长视频场景下，海量时空 Token 会显著增加计算成本



STAC: 面向长视频推理的时空 Token 压缩

- 思路：用线性复杂度的状态空间模型先建立时空上下文，再进行层级、自适应的 Token 压缩

现有方法：直接压缩原始特征



缺少时空上下文
压缩决策主要依赖表层相似度

可能误删关键运动线索
或目标边界信息

核心矛盾

要判断哪些 token 冗余，需要先获得时空上下文；但完整上下文建模本身又会带来高计算成本。

STAC: 状态感知压缩



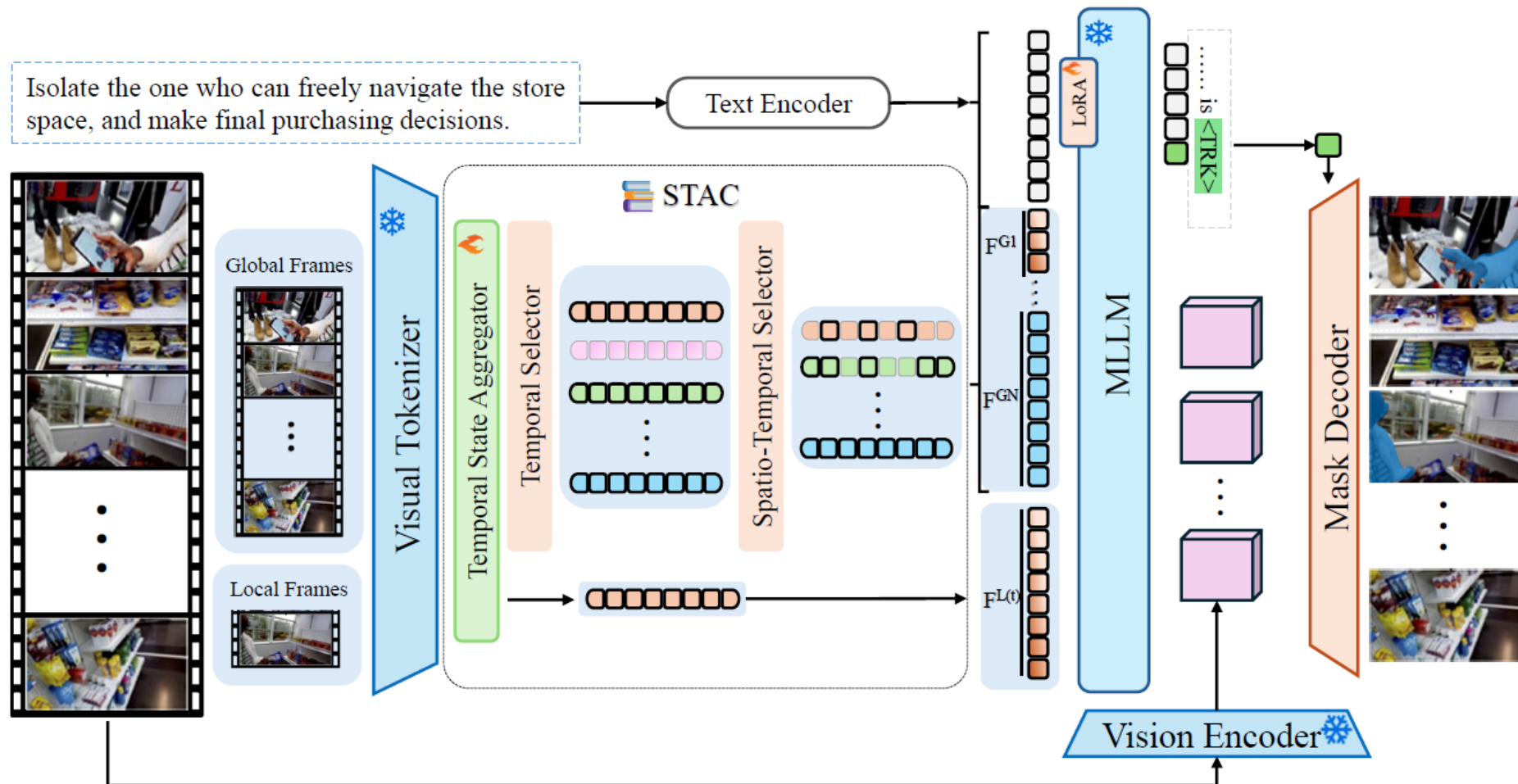
SSM先累积时空状态
让冗余信号可被识别

再按任务目标保留
对分割有用的Token

对比：STAC 同时支持时序建模、任务感知压缩、层级理解与在线处理。

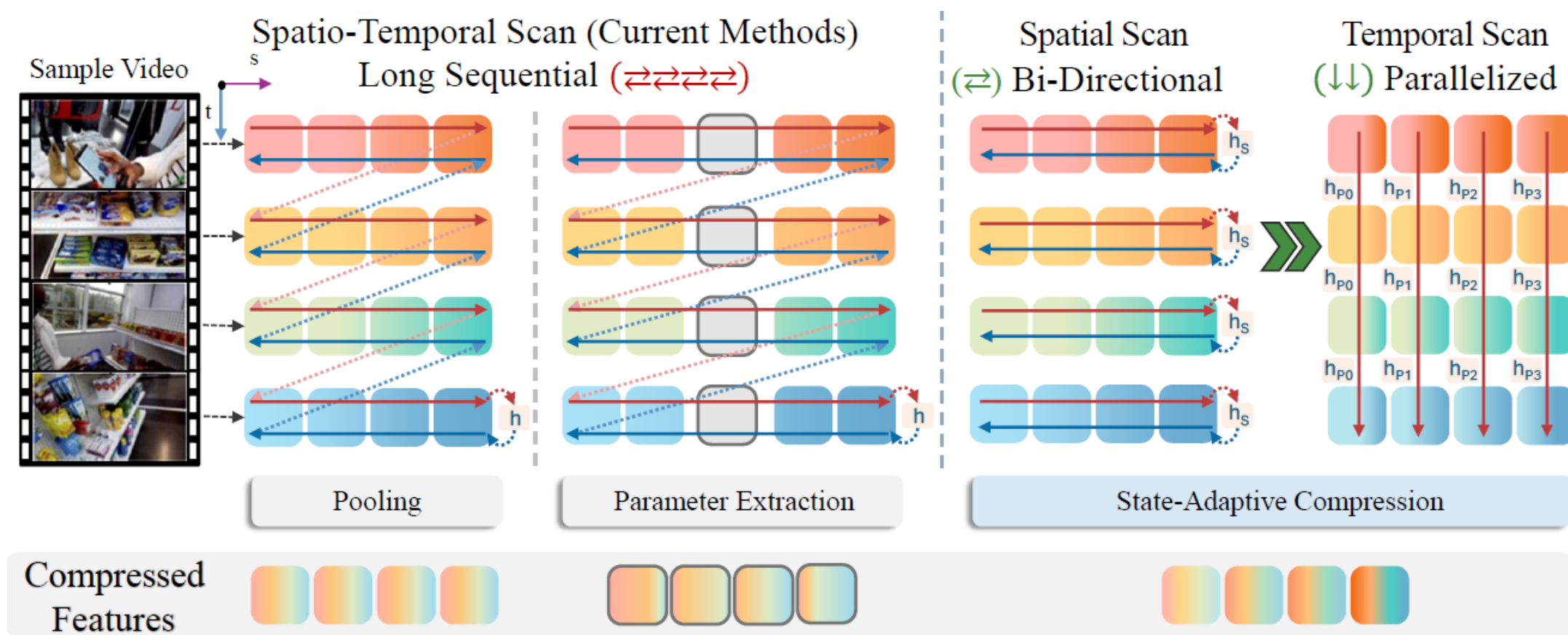
STAC: 面向长视频推理的时空 Token 压缩

- STAC 首先利用状态空间模型对视频 token 进行时空上下文增强，再通过时间—空间层级压缩减少冗余 Token



STAC: 面向长视频推理的时空 Token 压缩

- STAC 不将视频展平成单一长序列统一处理，而是根据空间与时间的不同属性，采用解耦的扫描与压缩策略



STAC: 面向长视频推理的时空 Token 压缩

- 层级自适应压缩：先时间，再空间
 - 根据运动变化与分割目标自适应保留关键 token

核心机制

1 时间压缩
相邻帧相似 → 合并冗余历史
保留运动变化明显的关键帧

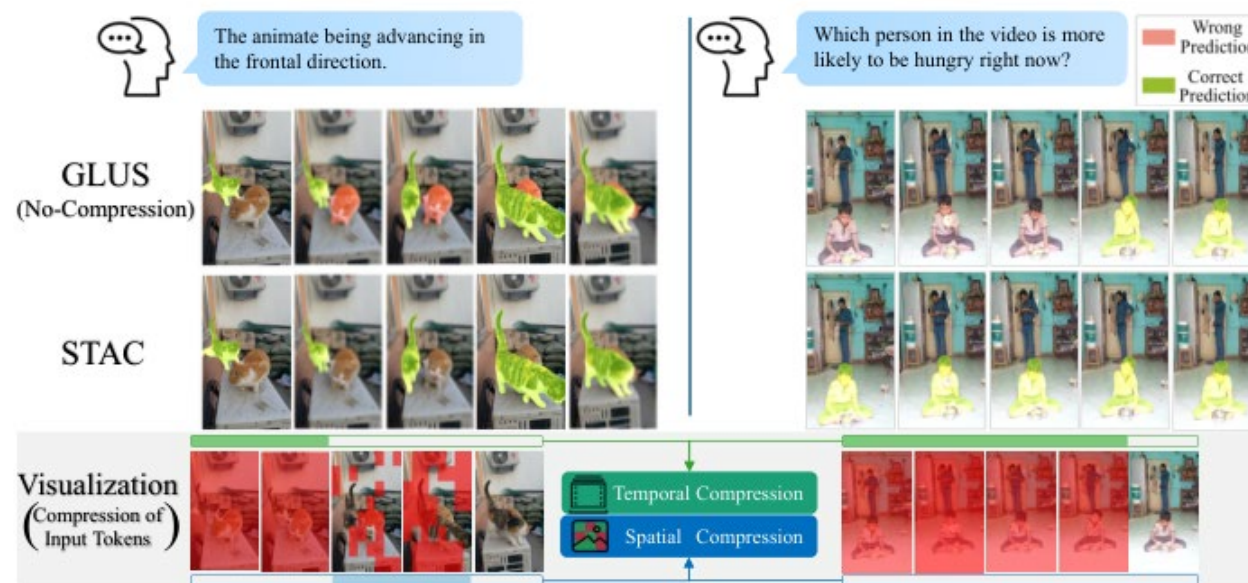


2 空间压缩
静态背景区域 → 强压缩
动态目标区域 → 多保留



3 任务监督优化
分割损失反向约束压缩阈值
保留对 mask 最关键的信息

自适应保留关键区域



红色区域表示被压缩的帧或 patch

静态区域压缩更多，运动关键区域保留更多

STAC: 面向长视频推理的时空 Token 压缩

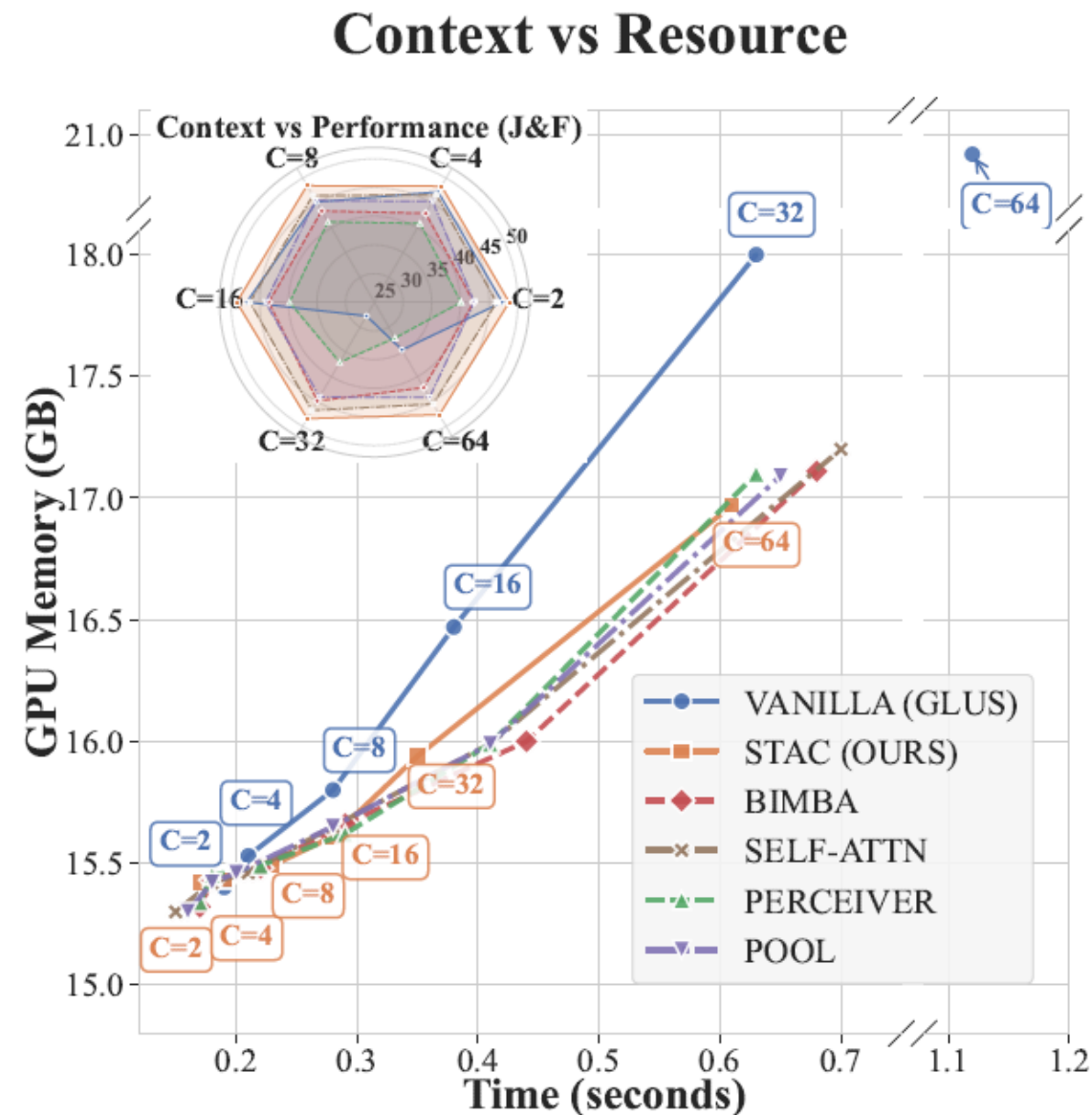
- 现有方法往往在“时序建模、任务感知压缩、层级理解、在线处理”之间难以兼顾
- STAC 通过状态感知的时空聚合与压缩，在保持推理分割能力的同时支持长视频高效处理

Method	Temporal Integration	Task-Aware Compression	Hierarchical Understanding	Online Processing
<i>Local Compression Methods</i>				
FastV [11]	✗	✗	✗	✗
ATP-LLaVA [80]	✗	✓	✗	✗
TempMe [65]	✓	(✓)	✓	✗
<i>Adaptive Compression Methods</i>				
LongVU [66]	✓	✓	✗	✗
M-LLM [31]	✓	✓	✗	✗
TSPO [68]	✓	✓	✗	✗
<i>Bidirectional SSM Methods</i>				
VideoMamba [43]	✓	✗	✗	✗
BIMBA [32]	✓	(✓)	✗	✗
STORM [35]	✓	✗	✗	✗
STAC (Ours)	✓	✓	✓	✓

STAC: 面向长视频推理的时空 Token 压缩

- STAC 在不同视频长度下，兼顾效率和性能

- 随着帧数增加，未压缩方法的显存和时延快速上升；
- STAC 在不同上下文长度下保持稳定的 J&F 表现；
- 相比其他压缩策略，STAC 在资源消耗和分割性能之间取得更优平衡



STAC: 面向长视频推理的时空 Token 压缩

- STAC 仅需处理15%的视觉 Token，而其他方法需处理全部

Models	Ref-DAVIS17			MeViS			Ref-YouTube			ReasonVOS			ReVOS		
	$\mathcal{J}\&\mathcal{F}$	$\mathcal{J}$	$\mathcal{F}$	$\mathcal{J}\&\mathcal{F}$	$\mathcal{J}$	$\mathcal{F}$	$\mathcal{J}\&\mathcal{F}$	$\mathcal{J}$	$\mathcal{F}$	$\mathcal{J}\&\mathcal{F}$	$\mathcal{J}$	$\mathcal{F}$	$\mathcal{J}\&\mathcal{F}$	$\mathcal{J}$	$\mathcal{F}$
<i>Multimodal LLM-based Reasoning Methods</i>															
VideoGLaMM [54]	69.5	65.6	73.3	45.2	42.1	48.2	-	-	-	-	-	-	-	-	-
LISA-7B [41]	64.8	62.2	67.3	37.2	35.1	39.4	53.9	53.4	54.3	31.1	29.1	33.1	-	-	-
LISA-13B [41]	66.0	63.2	68.8	37.9	35.8	40.0	54.4	54.0	54.8	-	-	-	-	-	-
LLaMA-VID [44]+LMPM	-	-	-	-	-	-	-	-	-	-	-	-	26.1	20.9	31.4
VideoLISA-3.8B (1-Tok) [6]	67.7	63.8	71.5	42.3	39.4	45.2	61.7	60.2	63.3	45.1	43.1	47.1	-	-	-
VideoLISA-3.8B (Post) [6]	68.8	64.9	72.7	44.4	41.3	47.6	63.7	61.7	65.7	47.5	45.1	49.9	-	-	-
TrackGPT-7B [85]	63.2	59.4	67.0	40.1	37.6	42.6	56.4	55.3	57.4	-	-	-	43.6	41.8	45.5
TrackGPT-13B [85]	66.5	62.7	70.4	41.2	39.2	43.1	59.5	58.1	60.8	-	-	-	45.0	43.2	46.8
VISA-Chat-7B [78]	69.4	66.3	72.5	43.5	40.7	46.3	61.5	59.8	63.2	-	-	-	46.1	43.9	48.2
VISA-Chat-13B [78]	70.4	67.0	73.8	44.5	41.8	47.1	63.0	61.4	64.7	-	-	-	47.5	45.3	49.7
GLUS ^S [46]	71.9	68.3	75.4	50.3	47.5	53.0	64.8	63.3	66.4	49.8	47.2	52.5	49.5	46.9	52.0
STAC (Ours)	73.6	70.1	77.2	49.9	47.2	52.5	65.2	63.5	66.9	52.3	49.5	55.0	48.4	45.8	51.3

STAC: 面向长视频推理的时空 Token 压缩

- 消融实验

- 扫描方向、压缩策略；扫描和压缩顺序

Spatial Scan	Temporal Scan		Prune	Merge	ReasonVOS		
	Uni-Dir	Bi-Dir			$\mathcal{J}\&\mathcal{F}$	$\mathcal{J}$	$\mathcal{F}$
X	X	X	X	X	48.1	45.4	50.0
✓	X	X	X	✓	44.1	41.0	47.1
✓	X	✓	X	✓	42.7	39.4	46.1
X	✓	X	X	✓	45.2	42.2	48.3
✓	✓	X	X	✓	49.6	47.2	52.0
✓	✓	X	✓	X	47.9	45.4	50.4

Scan		Comp.		ReasonVOS		
Sp. 1st	Tmp. 1st	Sp. 1st	Tmp. 1st	$\mathcal{J}\&\mathcal{F}$	$\mathcal{J}$	$\mathcal{F}$
	✓	✓		31.3	27.5	35.2
	✓		✓	29.6	25.4	33.9
✓		✓		45.7	43.4	48.0
✓			✓	49.6	47.2	52.0

STAC: 面向长视频推理的时空 Token 压缩

• 未来工作

- 更长视频与在线推理
 - 面向更长时序、更复杂场景的视频输入，进一步提升流式处理中的稳定性与长程依赖建模能力
- 状态容量与压缩边界
 - 研究状态空间模型的状态容量、视频复杂度与可压缩比例之间的关系，建立更可解释的压缩准则
- 多模态流扩展
 - 从视频 token 压缩扩展到音频-视频、语言-视频、机器人感知等多模态连续输入场景，支持真实无人系统中的高效感知与决策

Code: <https://github.com/MCG-NKU/nku-video>

汇报 提纲

—
CONTENTS

背景与挑战

长视频时空压缩_{STAC}

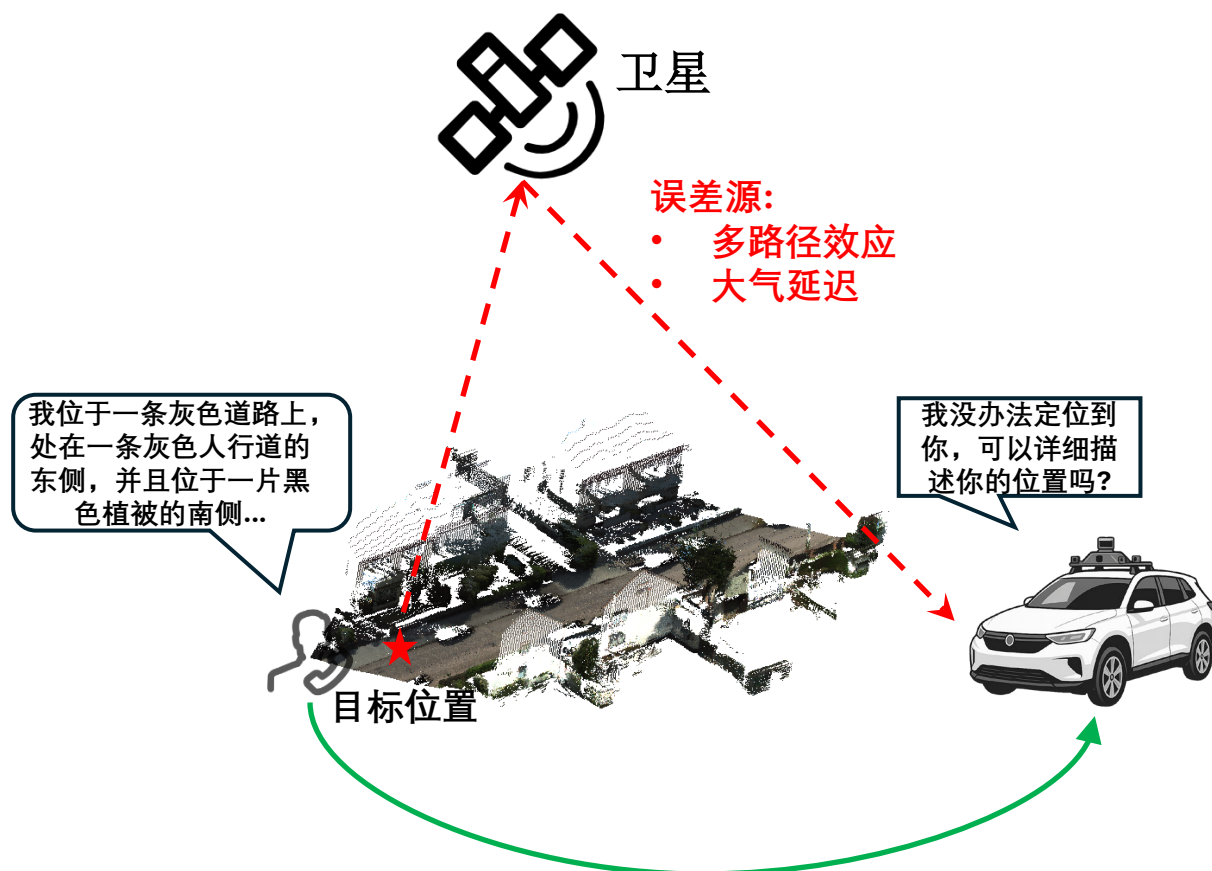
三维空间推理_{VLM-Loc}

端侧低比特部署_{COD-TDQ}

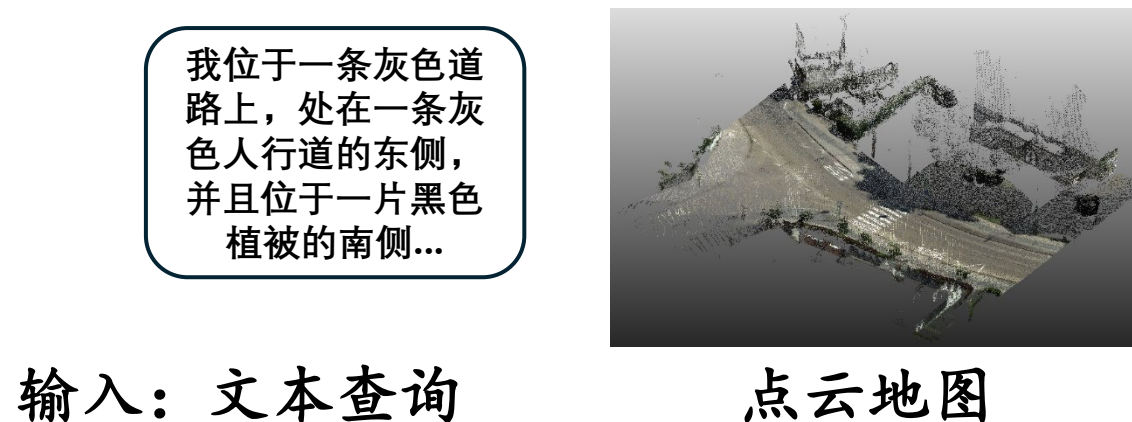
S. Kang, Y. Liao, P. Wang, W. Liao, Q. Zhang, B. Busam, X. Chen, and Y. Liu. VLM-Loc: Localization in Point Cloud Maps via Vision-Language Models. IEEE CVPR 2026.

VLM-Loc: 基于视觉语言模型的点云地图定位算法

• 任务设定: 文本在点云地图中的定位



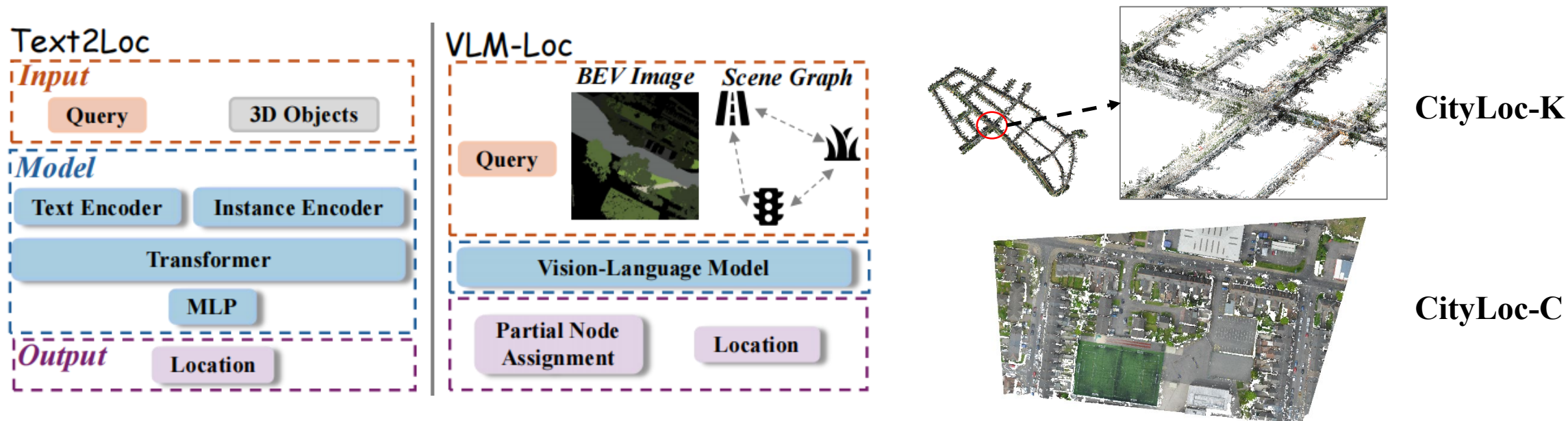
GNSS定位和文本在点云地图中的定位
对比示意图



VLM-Loc: 基于视觉语言模型的点云地图定位算法

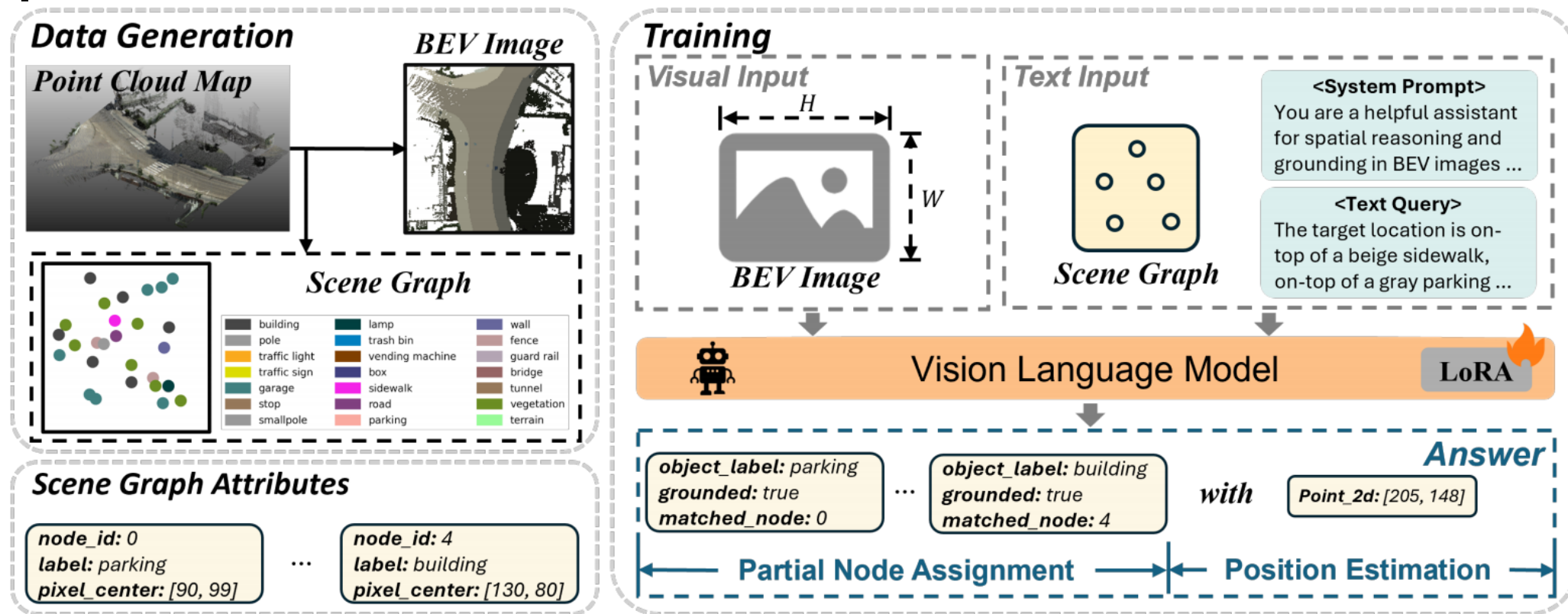
- 面临挑战
- ◆ 地图尺度受限 - 现有地图覆盖范围小，文本描述与点云实例之间的匹配难度低于真实场景；
 - ◆ 端到端定位可解释性不足 - 先前方法大多采用端到端框架，难以清晰揭示定位推理过程；
 - ◆ 跨域泛化能力有限 - 现有数据集场景类型较为单一，对未知场景的泛化能力不足。

- 解决方案
- ◆ 构建基于视觉语言模型的定位框架，在统一推理链条中同时输出匹配结果与定位结果；
 - ◆ 将原始点云表示转换为鸟瞰图与场景图联合输入，兼顾全局空间信息与结构化语义信息；
 - ◆ 构建更大空间范围的激光雷达文本数据集，并结合摄影测量点云测试集开展跨域泛化评测。



VLM-Loc: 基于视觉语言模型的点云地图定位算法

• 流程



- 将点云转换为鸟瞰图和场景图表示；
- 使用视觉语言模型分别处理视觉和语言输入；
- 将文本中的物体和场景图中的节点进行匹配，最后输出坐标。

VLM-Loc: 基于视觉语言模型的点云地图定位算法

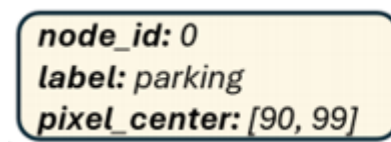
• 数据转换



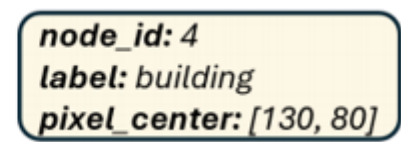
原始鸟瞰图



渲染后的鸟瞰图



...

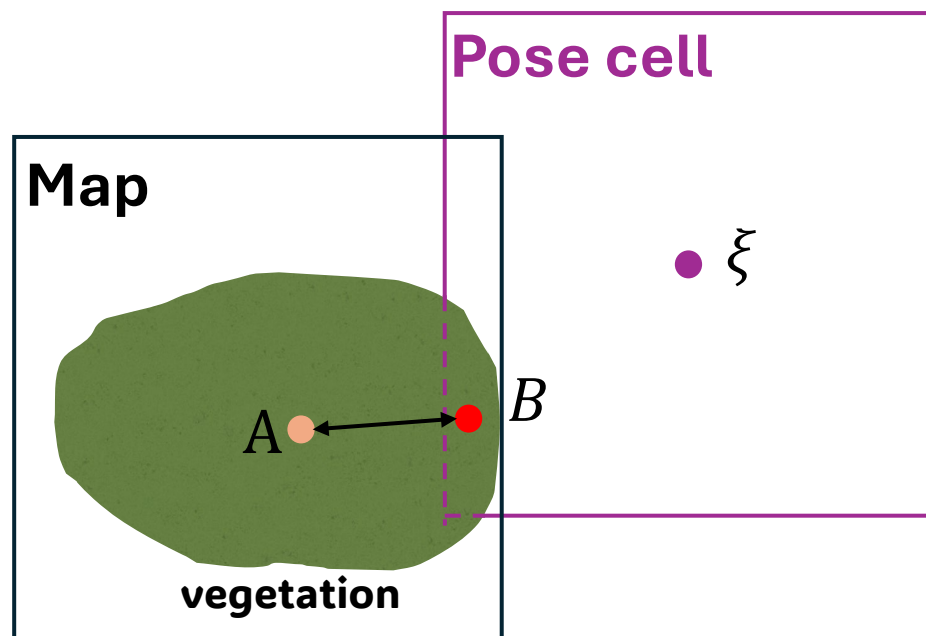


场景图

- 使用原始的点云RGB渲染鸟瞰图阻碍物体识别;
- 使用物体的平均颜色渲染;
- “可计数物体”实例绘制在“背景”类实例的上方以确保可见性
- 场景图由一组节点构成;
- 每个节点都被赋予一个节点 ID、一个语义标签, 以及一个以像素坐标表示的质心位置;
- 为简化表示, 边被省略。

VLM-Loc: 基于视觉语言模型的点云地图定位算法

• 部分节点分配策略



部分节点分配示意图

- Map cell 表示点云所覆盖的空间范围；
- Pose cell 定义了生成文本查询时可见的局部区域；
- 假设一个同时存在于 Map cell 和 Pose cell 中的植被实例；
- 设 A 和 B 分别为该植被在 Map cell 和 Pose cell 内点云的质心；
- 当 A 与 B 之间的距离超过特定阈值 τ 时，我们认为该节点是可锚定的。

VLM-Loc: 基于视觉语言模型的点云地图定位算法

• 消融结果

#	Input		Output	CityLoc-K Val	CityLoc-K Test
	BEV	SG	PNA	R@5/10/15m	R@5/10/15m
(a)	✓	✗	✗	13.04/32.79/52.31	13.21/33.86/51.40
(b)	✗	✓	✗	26.75/52.94/71.67	24.62/51.25/69.46
(c)	✗	✓	✓	<u>33.69/59.48/75.51</u>	<u>32.34/61.34/74.94</u>
(d)	✓	✓	✗	29.06/59.37/ <u>77.01</u>	29.79/57.57/73.78
(e)	✓	✓	✓	36.23/63.66/77.77	35.91/63.81/76.79

不同组件组合的定位结果

Model	CityLoc-K Val	CityLoc-K Test
	R@5/10/15 m	R@5/10/15 m
Qwen3-VL-2B-Instr.	35.78/64.22/79.97	34.70/63.19/76.49
Qwen3-VL-4B-Instr.	35.50/64.28/80.76	34.23/61.37/75.18
Qwen3-VL-32B-Instr.	39.84/67.72/82.05	41.05/67.47/79.39
InternVL3.5-8B [55]	<u>38.32/65.41/81.21</u>	<u>38.14/63.66/77.25</u>
Qwen3-VL-8B-Instr.	36.23/63.66/77.77	35.91/ <u>63.81/76.79</u>

不同类别与参数规模的VLM的定位结果

#	Components			CityLoc-K Val	CityLoc-K Test
	Sem.	Color	Dire.	R@5/10/15m	R@5/10/15m
(a)	✓	✗	✗	<u>18.74/43.17/63.15</u>	16.93/40.81/60.22
(b)	✓	✓	✗	18.28/ <u>43.96/64.50</u>	<u>18.01/42.95/61.47</u>
(c)	✓	✓	✓	36.23/63.66/77.77	35.91/63.81/76.79

不同文本组成方式的定位结果

VLM-Loc: 基于视觉语言模型的点云地图定位算法

• 实验结果 - 1

Method	CityLoc-K Val			CityLoc-K Test		
	R@5 m	R@10 m	R@15 m	R@5 m	R@10 m	R@15 m
Text2Pos [25]	16.48	40.69	62.92	14.62	38.27	59.55
Text2Loc [57]	18.91	45.26	64.28	17.97	41.22	61.50
MNCL [33]	19.30	45.94	64.50	18.76	42.63	62.58
CMMLoc [58]	<u>20.77</u>	<u>48.65</u>	<u>67.89</u>	<u>21.71</u>	<u>46.67</u>	<u>66.00</u>
VLM-Loc	36.23 (+15.46)	63.66 (+15.01)	77.77 (+9.88)	35.91 (+14.20)	63.81 (+17.14)	76.79 (+10.79)

VLM-Loc与基线方法在CityLoc-K的定位结果

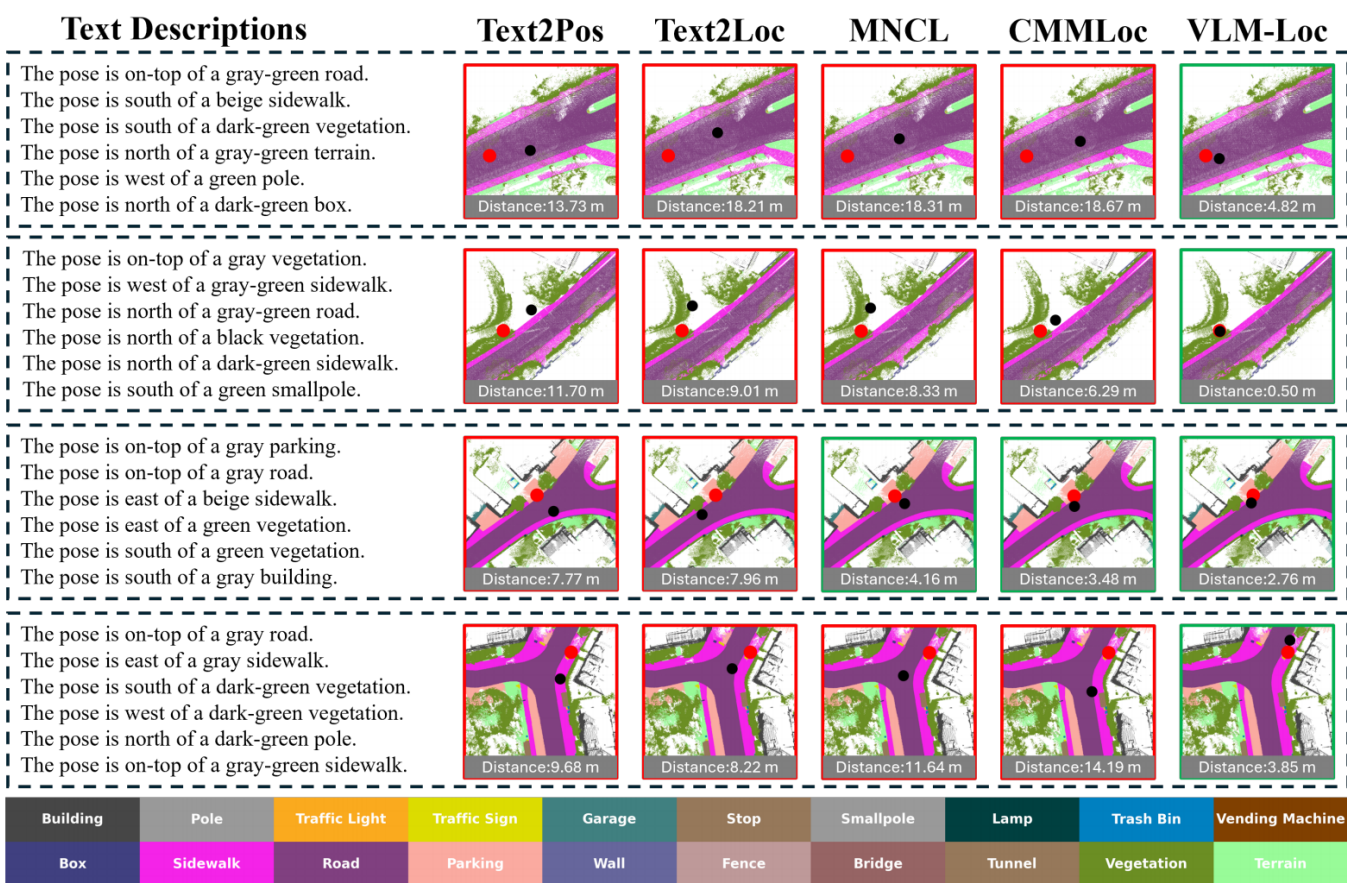
Method	CityLoc-C		
	R@5 m	R@10 m	R@15 m
Text2Pos [23]	8.11	27.21	50.01
Text2Loc [51]	9.45	29.44	51.17
MNCL [30]	<u>13.68</u>	<u>35.93</u>	53.78
CMMLoc [52]	11.68	34.79	<u>54.71</u>
VLM-Loc	21.37	49.12	68.26

在CityLoc-C的泛化结果

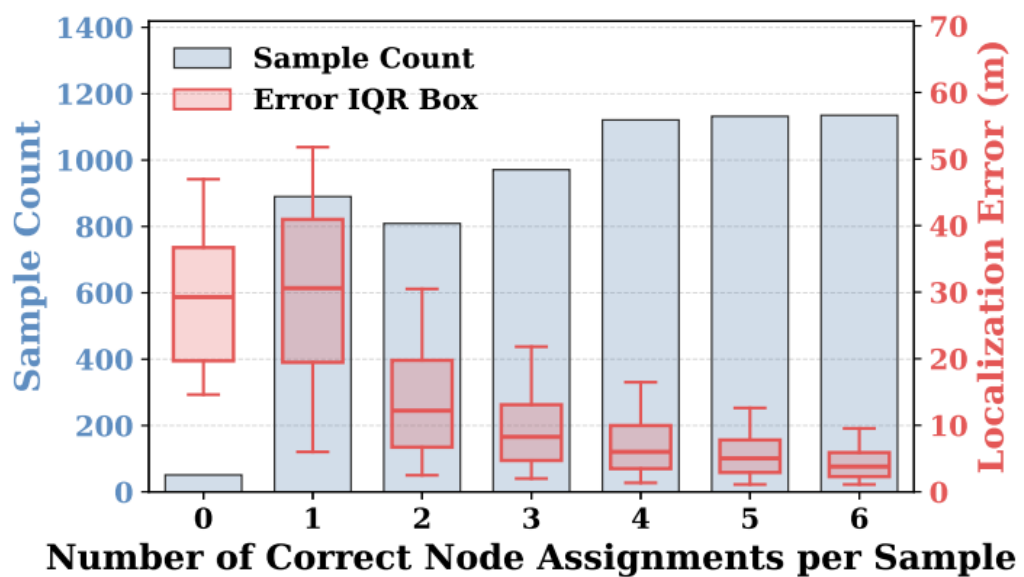
- VLM-Loc在CityLoc-K和CityLoc-C上均取得最好的定位结果；
- 尽管在泛化测试上精度下降，仍然显著领先所有的对比方法。

VLM-Loc: 基于视觉语言模型的点云地图定位算法

• 实验结果 - 2



VLM-Loc与基线方法的定性结果



节点分配结果统计图

- 大多数样本都可以预测3个及以上正确的对应；
- 节点预测正确的数量越多，定位误差越小且分布更集中。

VLM-Loc: 基于视觉语言模型的点云地图定位算法

• 未来工作

- 大尺度地图下的文本定位
 - 面向更大范围、更复杂的城市级环境，提升方法在多区域、多场景变化条件下的稳定性与泛化能力
- 交互式智能定位与持续修正机制
 - 支持多轮交互式输入与反馈修正，使定位过程由单次推理扩展为持续优化过程
- 面向实际应用的端到端系统能力
 - 推动从“定位任务”向“定位+决策+导航”的一体化系统演进，增强在真实机器人与自动驾驶场景中的可用性

Code: <https://github.com/MCG-NKU/nku-3d-vision>

汇报 提纲

—
CONTENTS

背景与挑战

长视频时空压缩_{STAC}

三维空间推理_{VLM-Loc}

端侧低比特部署_{COD-TDQ}

T. Li, W. Fang, X. He, X. Geng, X. Cheng, and Y. Liu. When W4A4 Breaks Camouflaged Object Detection: Token-Group Dual-Constraint Activation Quantization. ECCV 2026.

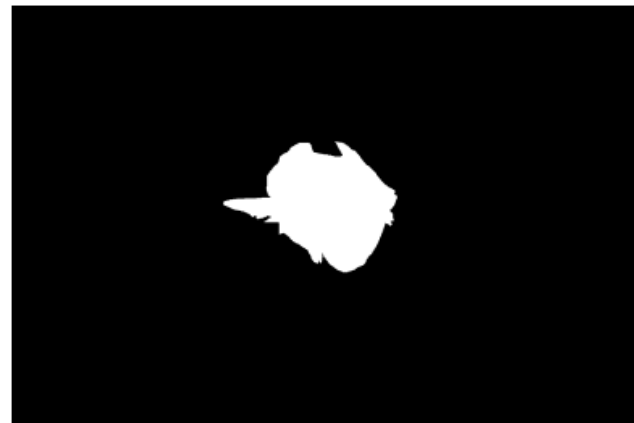
COD-TDQ: 伪装目标检测模型的W4A4量化

- COD-TDQ: 让W4A4不再“看不见”伪装目标

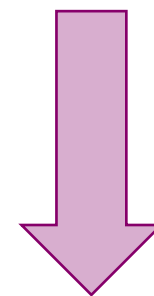
Input



FP32



端侧伪装物体检测 (Camouflaged Object Detection) 低比特部署



Naive W4A4



COD-TDQ



省成本，不牺牲边界感知

COD-TDQ: 伪装目标检测模型的W4A4量化

• 关键问题——为什么要做W4A4?

移动端部署希望降低存储和带宽

弱边界、细纹理结构响应敏感



低比特量化具有明显吸引力

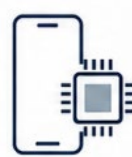
高精度需求

高部署成本

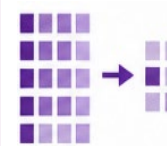
低比特量化(W4A4)



依赖细纹理和边界
对量化误差更敏感



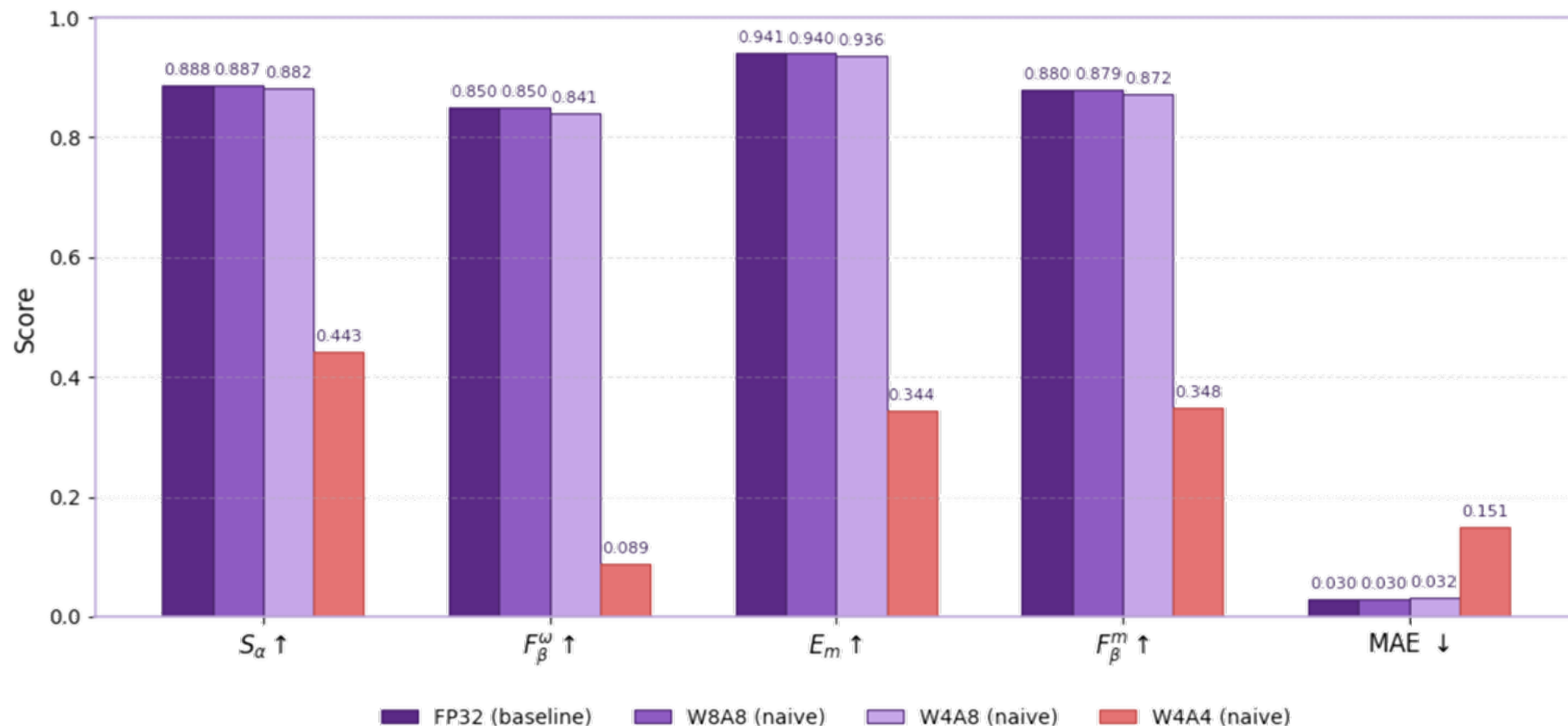
端侧部署受限
算力功耗压力大



W4A4显著压缩模型
降低存储与计算开销

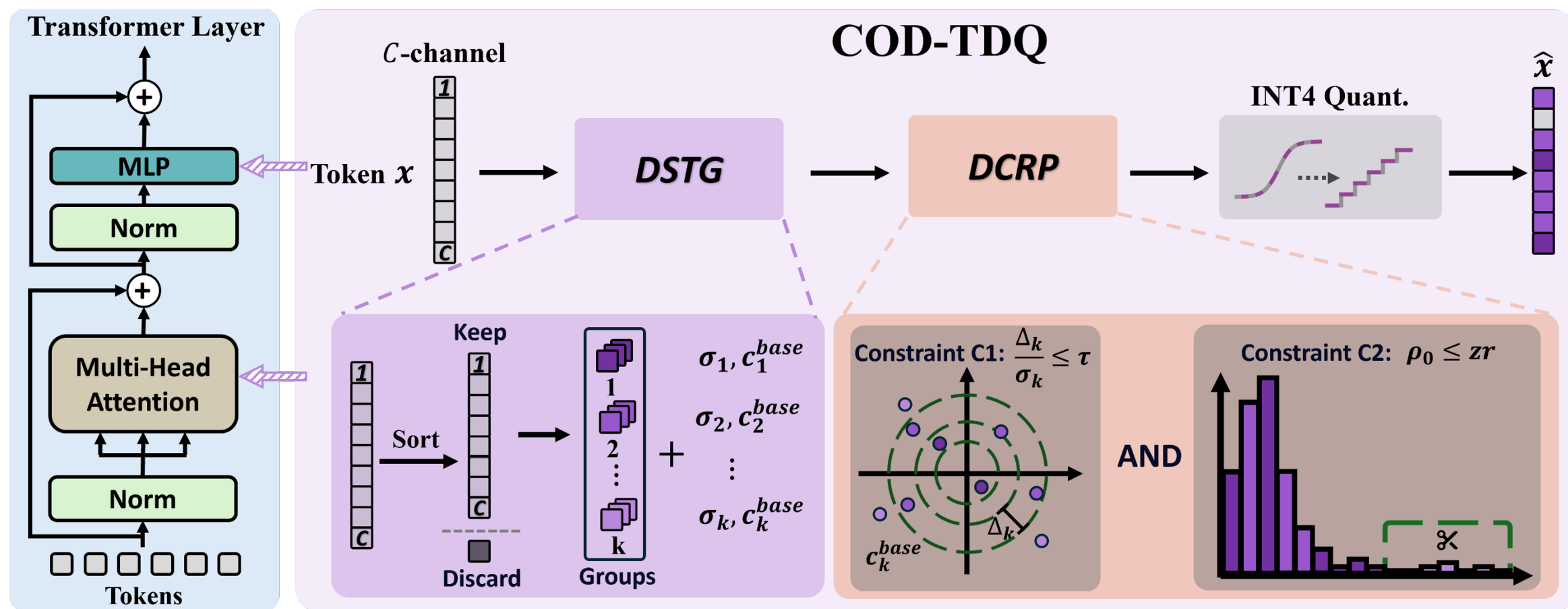
COD-TDQ: 伪装目标检测模型的W4A4量化

- 异常现象——W4A4不是变差，是崩溃
 - W4A8接近FP32，但W4A4 S_α 从0.888跌到0.443



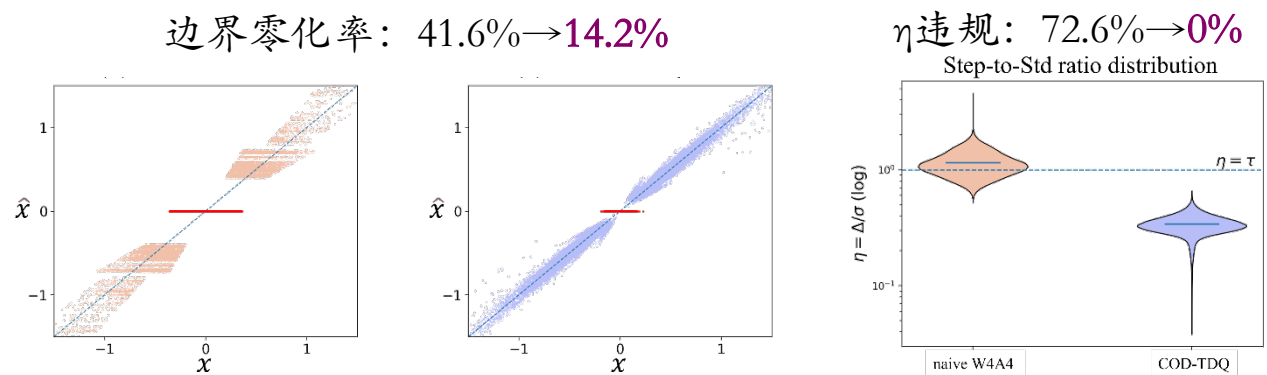
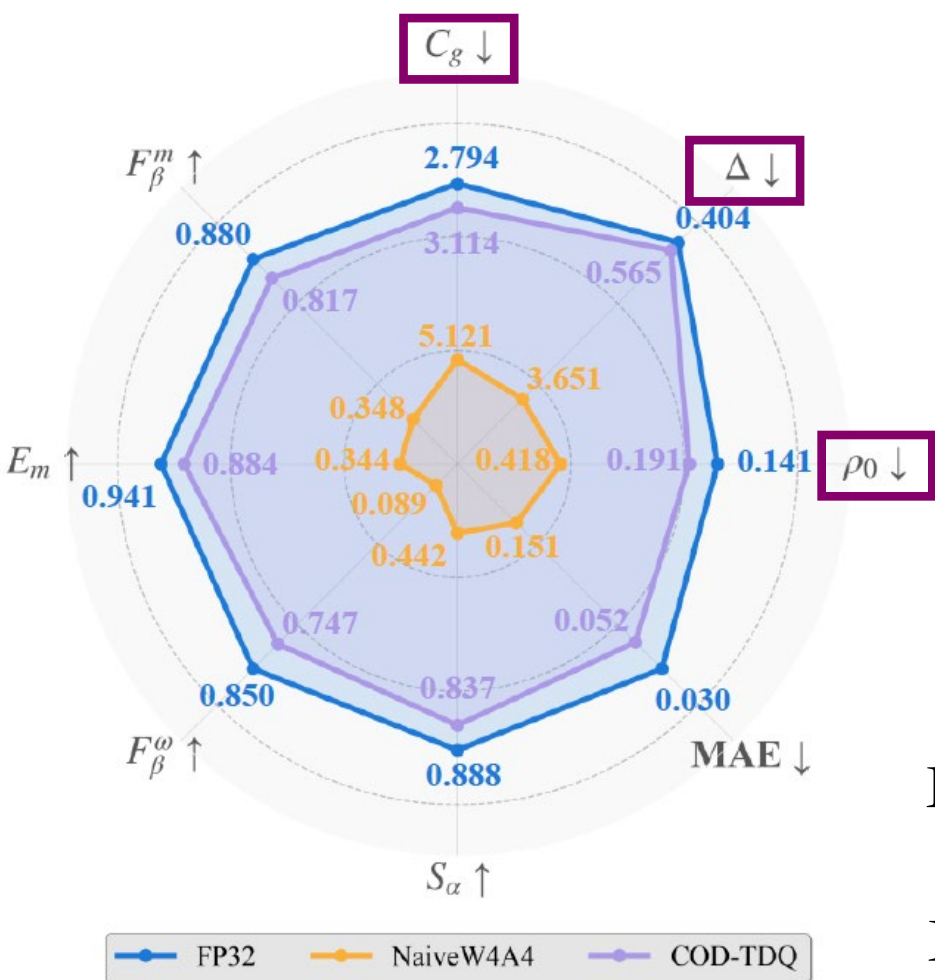
COD-TDQ: 伪装目标检测模型的W4A4量化

- 解决思路——COT-TDQ的“token-group受控量化”
 - DSTG分组控范围+ DCRP 约束零化



COD-TDQ: 伪装目标检测模型的W4A4量化

- 有效机制——直接压住两个崩溃指标



约束 C1: 控制步长离散度比

$$\eta_k \triangleq \frac{\Delta_k}{\sigma_k} \leq \tau \iff c_k \leq c_k^{(\tau)} \triangleq q_{\max} \tau \sigma_k.$$

约束 C2: 控制零桶质量映射

$$\rho_{0,k} \triangleq \frac{1}{g} \sum_{i=1}^g \mathbf{1}\left(|x_{k,i}| \leq \frac{\Delta_k}{2}\right), \quad c_k \leq c_k^{(\text{zr})} \triangleq 2q_{\max} Q_{\text{zr}}(|x_k|).$$

DSTG: 把 Token 沿通道切成若干组, 为各组分配局部量化尺度

DCRP: 把每组基础裁剪半径投影到满足两类约束的稳定区间

COD-TDQ: 伪装目标检测模型的W4A4量化

• 实际效果——精度显著恢复

Method	CAMO					CHAMELEON					COD10K					NC4K				
	S_α	F_β^ω	E_m	F_β^m	MAE	S_α	F_β^ω	E_m	F_β^m	MAE	S_α	F_β^ω	E_m	F_β^m	MAE	S_α	F_β^ω	E_m	F_β^m	MAE
Full Precision & Naive Quantization																				
FP32	.876	.844	.934	.877	.042	.912	.875	.961	.898	.019	.870	.795	.939	.835	.022	.888	.850	.941	.880	.030
Naive W8A8	.875	.843	.933	.876	.042	.912	.875	.961	.897	.019	.869	.795	.938	.834	.022	.887	.850	.940	.879	.030
Naive W4A8	.864	.826	.923	.861	.047	.906	.863	.959	.886	.021	.858	.776	.931	.818	.024	.882	.841	.936	.872	.032
Naive W4A4	.418	.061	.314	.332	.182	.447	.083	.351	.316	.141	.471	.071	.376	.242	.093	.443	.089	.344	.348	.151
Existing PTQ Methods																				
NoisyQuant	.408	.112	.498	.331	.190	.443	.120	.574	.318	.150	.474	.101	.380	.248	.095	.450	.173	.324	.341	.124
PTQ4ViT	.410	.061	.352	.318	.181	.440	.080	.380	.321	.140	.479	.070	.393	.252	.091	.456	.081	.340	.355	.153
ORQ-ViT	.415	.069	.303	.321	.181	.448	.083	.356	.316	.141	.489	.086	.364	.240	.092	.440	.081	.332	.344	.151
post-GELU	.403	.093	.469	.319	.185	.449	.084	.466	.318	.147	.480	.073	.544	.259	.084	.450	.094	.502	.352	.156
SmoothQuant	.397	.118	.526	.321	.140	.445	.104	.558	.328	.140	.490	.073	.565	.249	.091	.452	.108	.543	.342	.157
PQ-SAM	.464	.228	.511	.398	.121	.465	.190	.499	.317	.150	.483	.131	.462	.243	.089	.474	.211	.508	.351	.154
QuaRTZ	.416	.054	.309	.324	.182	.447	.083	.353	.319	.140	.471	.070	.371	.243	.093	.441	.084	.337	.350	.151
AHCPTQ	.417	.060	.313	.329	.182	.449	.086	.364	.320	.141	.471	.070	.374	.243	.093	.442	.086	.340	.345	.151
ARCQuant	.534	.510	.657	.629	.117	.593	.464	.619	.572	.096	.640	.455	.676	.570	.079	.683	.569	.711	.674	.097
FIMA-Q	.595	.549	.659	.553	.130	.603	.476	.682	.551	.096	.611	.452	.698	.518	.076	.663	.572	.733	.629	.089
PTQ4SAM	.671	.591	.740	.653	.106	.605	.461	.645	.518	.095	.665	.540	.760	.584	.072	.702	.632	.761	.660	.082
IGQ-ViT	.674	.601	.748	.658	.113	.692	.607	.763	.659	.069	.670	.538	.778	.592	.069	.710	.651	.776	.688	.080
RepQ-ViT	.676	.608	.750	.656	.099	.701	.622	.774	.672	.067	.676	.549	.779	.599	.062	.718	.651	.783	.691	.072
Ours	.813	.724	.864	.795	.070	.862	.758	.904	.823	.040	.802	.649	.864	.736	.038	.837	.747	.884	.817	.052

COD-TDQ 显著领先基线

已把 W4A4 从“不可用”拉回到“可部署”

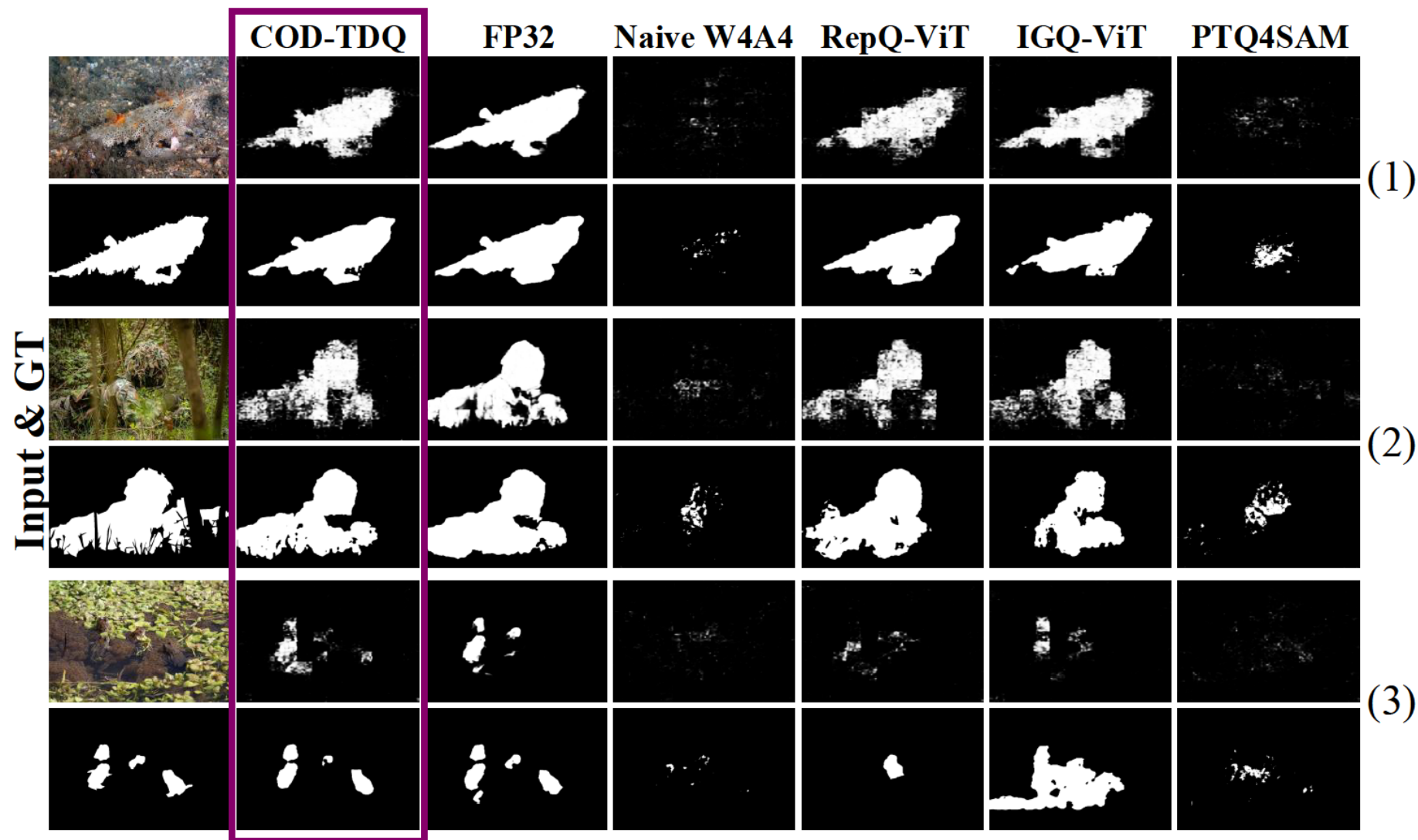
COD-TDQ: 伪装目标检测模型的W4A4量化

• 实际效果——精度显著恢复

• 三组输出定性展示

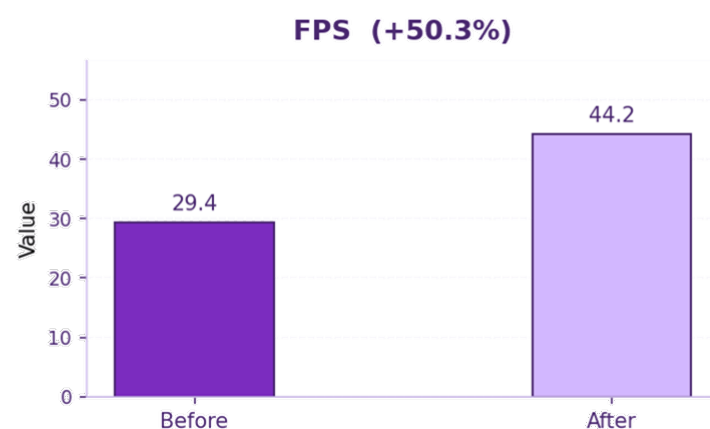
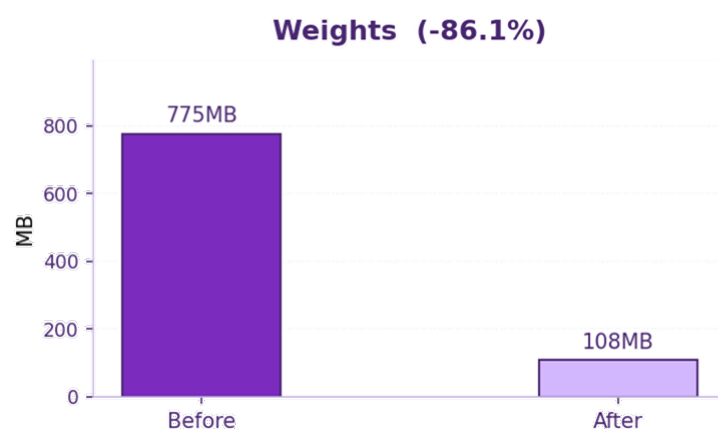
组内第一行CFRN为量化模型

组内第二行ESCNet为量化模型



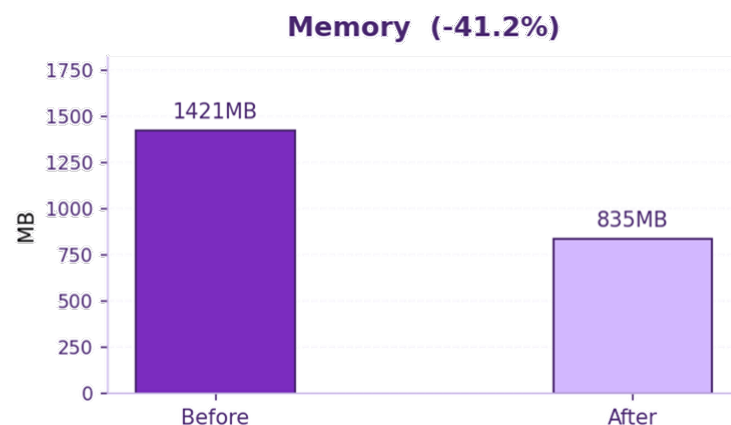
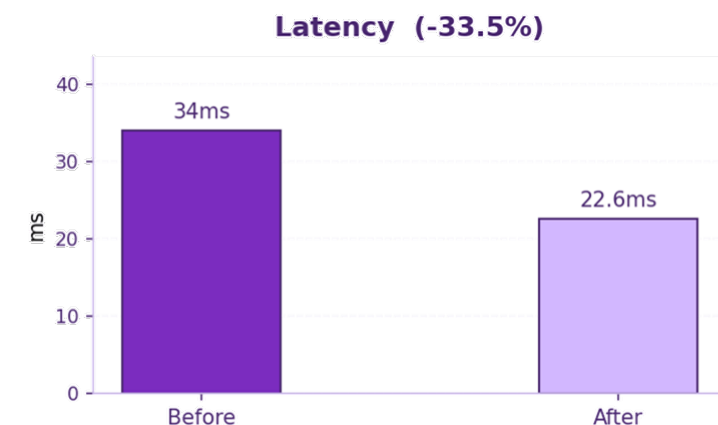
COD-TDQ: 伪装目标检测模型的W4A4量化

- 部署收益——更快、更小、更省显存



权重: 775MB→108MB

FPS: 29.4→44.2



延迟: 34.0ms→22.6ms

显存: 1421MB→835MB

机制可解释, 精度可恢复, 部署有收益

COD-TDQ: 伪装目标检测模型的W4A4量化

• 未来工作

- 边界敏感视觉任务扩展
 - 扩展到医学图像、遥感小目标和显著目标等弱边界视觉任务，提升低比特部署可靠性
- 跨架构与跨平台泛化
 - 验证方法在不同视觉模型、分割框架和基础模型中的适用性
- 硬件友好的自适应 INT4 部署
 - 结合真实端侧硬件推理约束，探索低成本自适应量化、误差诊断和 INT4 加速实现，推动从算法验证走向实际部署

Code: <https://github.com/MCG-NKU/nku-model-compre>

谢谢

https://yun-liu.github.io/materials/Slides_Efficient-MMVP.pdf